

Technical Documentation: Trello Power-Up "Blumenladen Produktliste"

1. System Overview

Das "Blumenladen Produktliste" Trello Power-Up ist eine clientseitige Web-Applikation, die als iframe innerhalb der Trello-UI (Atlassian Design System) gerendert wird. Das Power-Up ermöglicht es Nutzern, hierarchische Produktlisten (Hauptartikel und Unterartikel) auf Trello-Karten zu verwalten. Es bietet des Weiteren einen globalen Bestell-Manager, Katalogverwaltung, Lieferanten-Konfiguration und automatische Trello-Label-Synchronisation.

1.1 Architecture & Tech Stack

- **Architektur:** Iframe-basierte Micro-Frontend-Architektur. Das Power-Up läuft vollständig im Browser des Nutzers.
- **Frontend-Technologien:** Vanilla JavaScript (ES5/ES6), HTML5, CSS3. Keine schweren Frameworks (wie React/Vue) zur Wahrung der Performance.
- **Trello API:**
 - `TrelloPowerUp.initialize()` für Capability-Registrierung (`client.js`).
 - `TrelloPowerUp.iframe()` für die Kontext-Kommunikation innerhalb der iframes.
 - **Trello PluginData:** Zur Persistenz von Datenstrukturen (via `t.get()` und `t.set()`).
 - **Trello REST API:** (via `fetch`) zum Auslesen, Setzen und Löschen von Labels (`idLabels`).

2. Capability Endpoints (Manifest & Client)

Die Trello-Capabilities sind in der `manifest.json` deklariert und in der `client.js` implementiert:

Capability	Funktion	Referenzdatei	Beschreibung
card-buttons	"Produkte / Bestellung"	table.html	Öffnet das Modal auf einer bestimmten Karte, um Artikel (Haupt/Unter) hinzuzufügen.
board-buttons	"Bestell-Manager"	board.html	Board-weites Dashboard mit Filter-, Auswertungs- und CSV-Export-Funktionen.
board-buttons	"Katalog Manager"	katalog.html	Verwaltung der standardisierten Artikelvorlagen (Preise, Lieferanten).
show-settings	"Einstellungen"	settings.html	Verwaltung der dynamischen Lieferanten-Liste und Trello-Label-Mappings.

3. Data Storage & Schema (Trello PluginData)

Das Power-Up nutzt Trellos `shared` Scope zur Speicherung. Die Speicherung erfolgt asynchron auf den Servern von Atlassian/Trello.

3.1 produkte (Scope: card, Visibility: shared)

Speichert die auf der Karte angelegten Artikel.

```
[
  {
    "id": "180a5...",
    "stk": "2",
    "produkt": "Steckschaum",
    "preis": "10.50",
    "lieferant": "lief1",
    "unterartikel": [
      {
        "id": "180a6...",
        "stk": "1",
        "produkt": "Grün",
        "preis": "5.00",
        "lieferant": "lief2",
        "status": "bestellen" // Enum: "bestellen", "zulauf", "vorhande
      }
    ]
  }
]
```

3.2 lieferanten (Scope: board, Visibility: shared)

Speichert die dynamische Liste der konfigurierten Lieferanten für das Board.

```
[
  {
    "id": "lief1",
    "name": "Volmary",
    "labelId": "5e1f7a..." // Verknüpfte Trello-Label-ID
  },
  {
    "id": "lief_1722638848123",
    "name": "Kientzler",
    "labelId": "5e1f7b..."
  }
]
```

3.3 katalog (Scope: board, Visibility: shared)

Speichert die standardisierten Artikelvorlagen.

```
[
  {
    "name": "Blumendraht",
    "preis": "2.99",
    "lieferant": "lief1"
  }
]
```

4. Core Modules & Business Logic

4.1 table.js (Karten-Ansicht)

- **Verantwortlichkeit:** Rendern der Tabelle innerhalb einer Karte. CRUD-Operationen für Haupt- und Unterartikel.
- **Validierung:** Strikte Integer-Validierung für Stückzahlen (`Number.isInteger`). Sicheres Parsing von Float-Preisen (`inputmode="decimal"`, , zu . Normalisierung).
- **Trigger:** Löst nach jedem `t.set` die `syncCardLabels()` Methode (aus `utils.js`) aus.

4.2 board.js (Bestell-Manager & Auswertung)

- **Verantwortlichkeit:** Board-weite Aggregation aller Karten via `t.cards('all')`.
- **Zustandsverwaltung:** Persistiert Filter (Suchtext, Status, Lieferant, ViewMode) im Browser `sessionStorage` (`blumenladen_filter_zustand`), mit Ausnahme des Datums ("Von"), welches immer auf den aktuellen Tag (heute) forciert wird.
- **Race-Condition-Schutz (`updateBestellStatus`):** Verhindert redundante API-Requests durch schnelles Klicken mithilfe eines Set (`inFlightRequests`) und deaktivierten (`disabled`) Buttons.
- **Debouncing:** Nutzt einen 250ms Timeout beim Eingeben von Suchbegriffen (`filter-suche`), um UI-Blockaden zu verhindern.
- **Sicherheit (CSV-Export):** Nutzt `csvField()` für Quotation und das Voranstellen eines ' bei Formelzeichen (`=`, `+`, `-`, `@`), um Excel-Injections zu verhindern.

4.3 katalog.js (Katalogverwaltung)

- **Verantwortlichkeit:** Hinzufügen, Bearbeiten und Löschen von Katalogeinträgen.
- **Duplikat-Erkennung:** Strikter Abgleich per `trim().toLowerCase()`, um Duplikate wie "Rose rot" und "rose rot " zu verhindern.

4.4 settings.js (Dynamische Lieferanten)

- **Verantwortlichkeit:** Verwaltung der Lieferanten als dynamisches Array.
- **Migration:** Konvertiert automatisch ältere Datenstrukturen (`{ lief1: {...}, lief2: {...} }`) in das neue Array-Format `[...]`, wenn das Modal geöffnet wird.
- **Sicherheit:** Escaped sämtliche Benutzereingaben bei der DOM-Generierung der Dropdowns via `escapeHtml()`.

4.5 utils.js (Zentrale Helfer & API)

Beinhaltet systemweite Utilities, wie z.B. XSS-Schutz (`escapeHtml`), Währungsformatierung (`formatEuro`) und globale Fehlerbehandlung (`window.handleError`). **Trello Label-Sync (`syncCardLabels`)**:

- Holt Lieferanten (`t.get`) selbstständig.
 - Bestimmt aktivierte Lieferanten-IDs aus den Produkten und Unterartikeln der Karte (Sub-Status bestellen / zulauf).
 - Nutzt die Trello REST API (via `t.getRestApi().getToken()`), um die Labels asynchron (`Promise.all`) per POST und DELETE auf der `/1/cards/{id}/idLabels` Route an die Karte zu heften oder zu lösen.
-

5. UI / Styling (Atlassian Design System)

Die CSS-Dateien (`hell.css` und `dunkel.css`) sind strikt an das aktuelle **Atlassian Design System (ADS)** angelehnt.

- **Farben:** Nutzt `--trello-blue (#0C66E4)`, `--bg-subtle`, `--bg-hover`. Im Dark Mode werden diese Variablen in `dunkel.css` durch dunkle Pendants (z.B. `#579DFF`) überschrieben.
 - **Formen:** Radius für Buttons und Pills (Lozenges) sind fest auf `3px` limitiert, um der Trello-Kompaktheit zu entsprechen.
 - **Cache-Busting:** Alle CSS-Dateien werden in den HTML-Headern mit dem Parameter `?v=2` (o.ä.) eingebunden, um aggressive Browser-Caches auf der Trello-App/Webseite zu umgehen.
-

6. Security Considerations

- **Cross-Site Scripting (XSS):** User-Input (Produktnamen, Lieferanten-Namen) wird vor der DOM-Injektion (`.innerHTML` / `Template Literals`) konsequent mit der Funktion `escapeHtml` bereinigt.
 - **CSV Injection:** Vorbeugung durch explizite Typprüfung und Sanitizing in `board.js`.
 - **Authentication:** REST API-Calls erfolgen ausschließlich nach explizitem Trello-User-Consent (`t.getRestApi().isAuthorized()`). Der Access Token wird nie persistiert, sondern per API On-Demand abgerufen.
-

Generated by Antigravity IDE (2026).